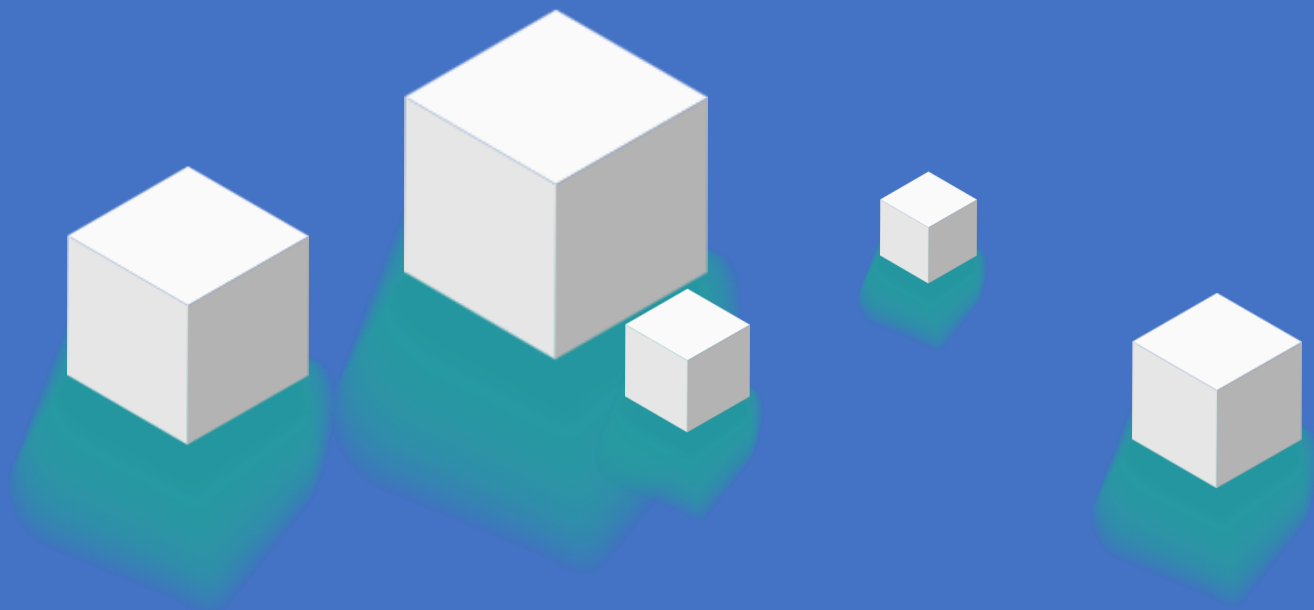


装备Talib技术指标库



>> 今天的学习目标

装备Talib技术指标库

- 什么是 TA-Lib?

CASE: Talib基础用法

TA-Lib的158个算子

- 经典指标的局限性

SMA/EMA, MACD, RSI, ATR

TA-Lib 的 61 种形态学算子

CASE: K线形态识别

锤子线, 射击之星、吞没形态, 十字星, 早晨之星

形态学在量化系统中的使用

- 传统指标增强

CASE: RSI策略-优化(穿越确认)

CASE: MACD策略-优化(成交量确认)

CASE: MACD策略-优化(利润锁定)

CASE: 布林带策略-优化(中轨止损)

CASE: 自适应策略

什么是 TA-Lib?

Thinking: Backtrader的因子计算是否够用?

Backtrader 是一个强大的事件驱动架构回测引擎 => 量化系统的大脑和调度中心

Backtrader 原生集成了移动平均 (SMA/EMA)、MACD、RSI 等主流技术指标, 在专业量化投研中, 这些指标够用么? 是否需要其他的因子?

=> 我们不一定要使用backtrader内置的计算逻辑。引入 TA-Lib (Technical Analysis Library) , 可能是更好的选择:

- 计算性能更优秀
- 观测指标更多元
- 数据解耦更方便

TA-Lib 计算性能更优秀

TA-Lib 计算性能更优秀：

- 实战中需要大量的指标计算

量化回测绝不仅仅是测试单只股票（如单测贵州茅台 600519.SH）。

在实战中，我们需要进行全市场截面分析——每天同时计算 A 股 5000 多只股票的 20 多个技术指标 => 构建多因子选股模型。

如果使用纯 Python 编写的内置指标（如 Pandas 的 `rolling.mean()` 或 Backtrader 的内建算子），由于 Python 在处理庞大循环时的机制问题，计算速度会呈指数级下降。

TA-Lib 计算性能更优秀

- TA-Lib 的底层是 C 语言编写的

通过 对比实验可以直观看到，在执行相同逻辑（如循环计算 1000 次 SMA20）时，TA-Lib 输出纯 Numpy 数组的速度比纯 Python 结构快数倍。

```
n_runs = 1000
start = time.time()
for _ in range(n_runs):
    talib.SMA(close, timeperiod=20)
talib_time = time.time() - start

start = time.time()
for _ in range(n_runs):
    pd.Series(close).rolling(20).mean()
pandas_time = time.time() - start
```

```
print(f" 计算SMA(20) x {n_runs}次:")
print(f" TA-Lib: {talib_time:.4f}s")
print(f" Pandas: {pandas_time:.4f}s")
print(f" TA-Lib 快 {pandas_time/talib_time:.1f} 倍")
```

[6] 计算速度对比

计算SMA(20) x 1000次:
TA-Lib: 0.0022s
Pandas: 0.0643s
TA-Lib 快 29.3 倍

1-Talib vs Backtrader对比.py

TA-Lib观测指标更多元

TA-Lib 观测指标更多元：

- 传统指标的盲区

绝大多数技术指标（均线、MACD、布林带）都属于滞后指标（它们是对历史价格序列的数学平滑）

⇒当 MACD 发生金叉时，市场往往已经上涨了一段空间

数学公式能过滤噪音，但也必然带来信号的延迟。

- TA-Lib 的技能：K线形态学

这是 Backtrader 内置指标库不具备的能力。

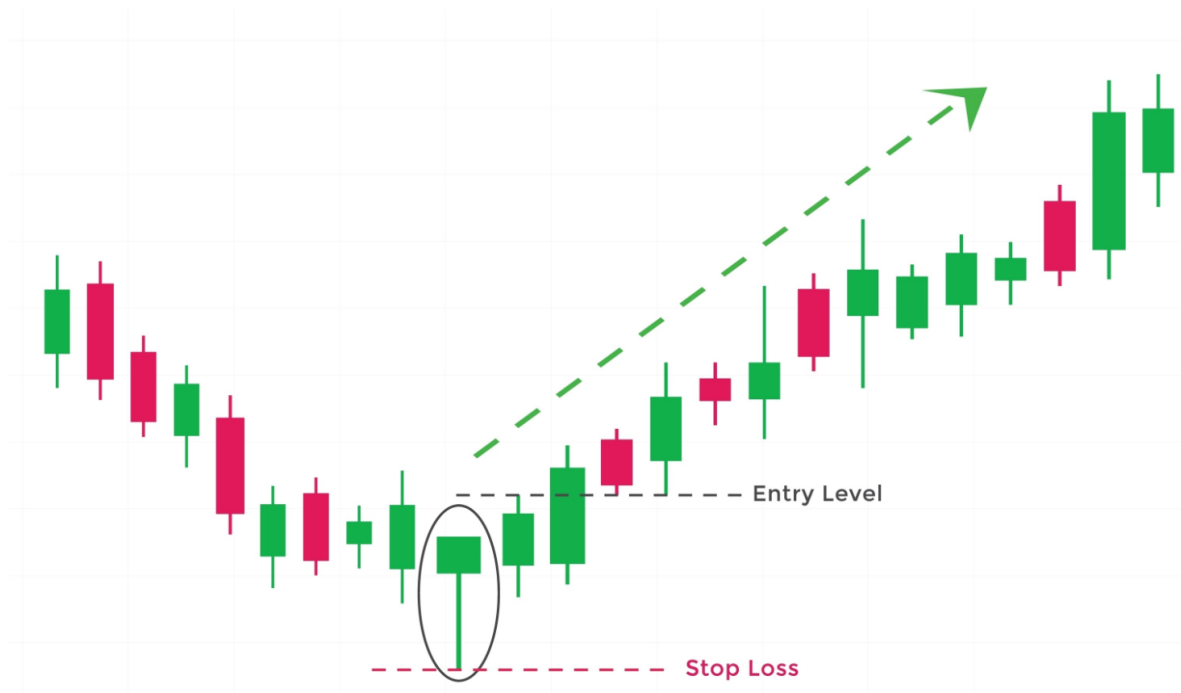
TA-Lib 内置了 61 种以 CDL_ 开头的形态识别函数。

TA-Lib观测指标更多元

Thinking: 如果在下跌趋势中, 出现一根实体极短、下影线极长 (比如是实体两倍以上) 的 K 线, 代表什么?

盘中多空双方激烈的刺刀见红。空头强力打压, 但多头在底部大举承接, 将价格硬生生推回开盘价附近。

以 CDLHAMMER (锤子线) 为例, 它不需要计算过去 20 天的均值, 而是直接扫描当下的 K 线结构



形态学捕捉的是当下的资金行为和情绪极值, 属于左侧或同步信号。

在实盘中, 可以将滞后指标 (判断大趋势方向) 与形态学 (捕捉情绪反转的入场点) 结合使用
=> 提升策略的胜率和盈亏比。

TA-Lib数据解耦更方便

TA-Lib 数据解耦更方便:

- Backtrader 的内置指标是与 `self.data` 数据流深度绑定的对象。

如果想提取这些指标的数值 => 作为特征向量喂给机器学习模型（如 XGBoost、LSTM、Transformer），提取过程繁琐，而且比较耗内存。

- TA-Lib 的输出极简

TA-Lib 的输入是极简的 Numpy 一维数组，输出也是极简的 Numpy 数组。

这种纯粹的 Array In, Array Out 模式，可以脱离任何回测框架，直接在数据清洗阶段（Data Pipeline）就用 TA-Lib 批量生成成百上千个动量、波动率特征（Feature Engineering）
=> 连接传统技术分析与现代 AI 量化预测模型之间的桥梁。

CASE: Talib基础用法

```
import numpy as np
import talib

print(f"TA-Lib 版本: {talib.__version__}")    # 0.6.8
print(f"可用函数数量: {len(talib.get_functions())}") # 158

# --- SMA 简单移动平均 ---
close = np.array([10, 11, 12, 13, 14, 15, 16, 17, 18, 19], dtype=np.float64)
sma5 = talib.SMA(close, timeperiod=5)
# 收盘价: [10. 11. 12. 13. 14. 15. 16. 17. 18. 19.]
# SMA(5): [nan nan nan nan 12. 13. 14. 15. 16. 17.]
# 手算验证: (10+11+12+13+14)/5 = 12.0 -> 一致

# --- MACD ---
macd, signal, hist = talib.MACD(prices, fastperiod=12, slowperiod=26,
signalperiod=9)
# DIF = EMA(12) - EMA(26), DEA = DIF的EMA(9), 柱状图 = DIF - DEA
```

```
# --- RSI ---
rsi = talib.RSI(prices, timeperiod=14)
# RSI > 70: 超买区 | RSI < 30: 超卖区

# --- ATR ---
atr = talib.ATR(high, low, close, timeperiod=14)
# 真实波幅 = max(High-Low, |High-PrevClose|, |Low-PrevClose|)
# 用途: 止损 = 入场价 - 2倍ATR
```

CASE: Talib基础用法

运行结果:

TA-Lib 版本: 0.6.8

可用函数数量: 158

指标类别:

Cycle Indicators	5 个
Math Operators	11 个
Momentum Indicators	30 个
Overlap Studies	17 个
Pattern Recognition	61 个
Volatility Indicators	3 个
Volume Indicators	3 个
...	

[SMA] 简单移动平均

收盘价: [10. 11. 12. 13. 14. 15. 16. 17. 18. 19.]

SMA(5): [nan nan nan nan 12. 13. 14. 15. 16. 17.]

手算验证: $(10+11+12+13+14)/5 = 12.0 \rightarrow$ 一致: True

TA-Lib的158个算子

TA-Lib 提供了 158 种算子

类别	代表指标	核心作用
均线类	SMA, EMA, WMA, DEMA, TEMA	定义市场重心与趋势轨道
动量类	RSI, MACD, MOM, CCI, ADX, STOCH	测量价格运动的加速度与动能衰竭
波动率类	ATR, NATR, TRANGE	衡量市场波动程度，动态风控的核心依据
成交量类	OBV, AD, ADOSC	验证价格突破的真实性，识别是否有真实资金推动
形态识别	CDLHAMMER, CDLENGULFING, CDLMORNINGSTAR	识别K线结构中的多空博弈信号
统计类	BETA, CORREL, LINEARREG, STDDEV	相关性与回归分析

TA-Lib提供的158个算子是经过几十年华尔街实盘验证的指标 => 帮助建立了完整的市场观测坐标系

经典指标的局限性

Thinking: 指标的作用是什么?

本质上是历史价格和成交量的时间序列滤波器 (Time-Series Filters) 。

通过不同的数学公式 (加和、平均、差值、比率) , 过滤掉市场日常的随机噪音

⇒ 凸显某种特定的市场特征 (如趋势、动量、波动率) 。

Thinking: 常用的经典指标有什么问题?

数学过滤都会带来一个致命的副产品: 信息折损与滞后性。

SMA / EMA (移动平均线)

SMA / EMA (移动平均线): 市场的平均持仓成本

- 简单移动平均 (SMA) 的公式为 $MA = \frac{P_1 + P_2 + \dots + P_n}{n}$

在信号处理中, 称为低通滤波器 (Low-pass Filter) => 抹平了短期的高频尖刺波动, 保留低频的趋势线。

- **博弈视角**: 均线代表了过去 n 天内, 市场参与者的平均持仓成本线。

如果当前价格 P0 位于 SMA(20) 之上时 => 过去 20 天买入的人整体处于盈利状态, 市场情绪偏向乐观。

- **致命局限**: 无解的滞后性与震荡市的来回打脸

因为是算术平均, 均线反应永远比价格慢半拍。当均线拐头向上时, 底部的利润往往已经错失了 10% 以上。

均线只能告诉你历史的重心, 无法预判未来的突破方向。

在单边趋势市中, 双均线策略是王者。但在横盘震荡市中, 均线会频繁发生无效交叉。

如果盲目跟随金叉死叉操作, 会产生大量摩擦成本 (手续费+滑点) => 续造成本金亏损。

MACD：价格运动的加速度与多空动能衰竭

MACD：价格运动的加速度与多空动能衰竭

- MACD是测算快慢两条指数均线的距离： $DIF = EMA(12) - EMA(26)$ 。

然后再对 DIF 进行一次平滑： $DEA = EMA(DIF, 9)$ 。

柱状图则是两者之差： $Histogram = DIF - DEA$ 。

- **博弈视角**：如果说均线测算的是价格的速度，那么 MACD 测算的就是价格的加速度。

金叉（DIF 上穿 DEA）并不代表绝对价格在涨，而是代表短期多方力量推升成本的速度，超过了长期的平均速度。

- **致命局限**：无量空涨与骗线陷阱MACD 的计算公式中完全没有纳入成交量（Volume）。

这意味着，只要价格被少量资金人为拉升（如尾盘偷袭），MACD 也会机械地画出一个完美的金叉。

在实战策略中，我们可以给MACD 打上成交量确认的补丁。如果发生金叉，但当天的成交量低于过去 20 日均量的 90%，有比较大的概率是缺乏资金真实推动的假信号，应当直接过滤放弃。

RSI：情绪的极限拉扯与均值回归陷阱

相对强弱指标 (RSI) 的公式为 $RS = \frac{\text{Average Gain}}{\text{Average Loss}}$

随后归一化至 0-100 区间： $RSI = 100 - \frac{100}{1+RS}$

- 博弈视角：这是多空双方的拔河比赛。

RSI 计算的是过去 n 天内，多方赢的幅度总和与空方赢的幅度总和的比例。

RSI > 70（超买区）意味着多方用力过猛，随时可能力竭；

RSI < 30（超卖区）则代表空方势能释放殆尽。

- 致命局限：单边市的严重钝化与接飞刀

有些新手看到 RSI 跌破 30 就以为是铁底，立刻买入。但在遇到系统性风险或主跌浪时，RSI 会在 10-20 的极度超卖区持续钝化（趴在底部不动），此时抄底就是接飞刀。

在实战策略中，我们可以引入穿越确认机制。即不在 RSI < 30 时立刻买入，而且等待 RSI 跌落底部后，重新向上穿过 30 的界限（rsi[-2] < 30 且 rsi[-1] >= 30）=> 代表多方真正开始反击，此时入场胜率会更好

ATR：市场的心率仪与降维风控

真实波幅 (True Range) 不仅计算当天的最高价减最低价，还考虑了昨收盘到今开盘的跳空缺口：

$$TR = \max(H - L, |H - P_{close}|, |L - P_{close}|)$$

ATR 就是 TR 的 N 日移动平均。

- **博弈视角：**它是市场的心率仪。

ATR 飙升，意味着市场处于极度恐慌或极度狂热的无序状态；

ATR 极低，则意味着市场处于无人问津的死水期。

- **致命局限：**它不提供方向。ATR 只能告诉你当前市场的振幅有多大，完全无法预判价格是向上涨还是向下跌。

绝大多数散户使用固定比例止损（例如亏损 5% 割肉）。但如果当前市场的 ATR 已经达到了 6%，那么 5% 的止损会被正常的市场日内波动轻易扫掉（被洗下车）。在 AI 量化中，使用动态波动率止损：将止损位设在入场价 - 2 * ATR。市场剧烈波动时，系统自动拉宽止损空间；市场平静时，系统自动收紧止损空间。

TA-Lib的K线形态学

- Thinking: AI量化是否需要看图识线?
- 传统指标的盲点

假设一只股票经历暴跌后，在底部发生了剧烈的多空搏杀，最终多头胜出，价格企稳反弹。

如果看均线或 MACD，由于之前累计的跌幅太大，均线可能还需要 3-5 天才会拐头、MACD 才会金叉。此时入场，最佳的抄底利润已经错失。

- 形态学的降维打击

K 线形态直接反映了当天的资金行为和情绪极值。

开盘价、收盘价、最高价、最低价的相对位置，构成了市场的微观博弈图。

=> TA-Lib将人们使用肉眼识别图形，严格转化为了一套量化数学规则（通过测算实体长度、上下影线比例的数学关系来判定）。

TA-Lib 的 61 种形态学算子与输出逻辑

我们展示了 TA-Lib 内置的 61 种以 CDL (Candle) 开头的函数。

- 计算机制

这些函数接受 Open, High, Low, Close 四个数组作为输入，输出一个与输入长度相同的整数数组。

- 信号规则

返回 +100：强烈的看涨形态（Bullish）。

返回 -100：强烈的看跌形态（Bearish）。

返回 0：未触发该形态。

CASE: K线形态识别

```
import numpy as np
import talib
from data_loader import load_stock_data

df = load_stock_data('600519.SH', '2024-01-01', '2025-12-31')
o = df['open'].values.astype(np.float64)
h = df['high'].values.astype(np.float64)
l = df['low'].values.astype(np.float64)
c = df['close'].values.astype(np.float64)

# 获取所有CDL开头的形态函数
cdl_funcs = [f for f in talib.get_functions() if f.startswith('CDL')]
# -> 61 种
```

```
# 逐个扫描, 统计出现次数
for func_name in cdl_funcs:
    func = getattr(talib, func_name)
    result = func(o, h, l, c)
    # result: +100 看涨 | -100 看跌 | 0 未触发

    bullish_count = np.sum(result > 0)
    bearish_count = np.sum(result < 0)
    # ...
```

CASE：K线形态识别

运行结果（扫描茅台 600519.SH，2024-2025 年共 481 个交易日）：

TA-Lib K线形态函数: 61 种

看涨形态 (共 32 种出现过):

- 十字星 (CDLDOJI) 出现 64 次, 最近: 2025-12-26
- 纺锤线 (CDLSPINNINGTOP) 出现 49 次, 最近: 2025-12-19
- 长实体 (CDLLONGLINE) 出现 35 次, 最近: 2025-11-13
- 捉腰带线 (CDLBELTHOLD) 出现 31 次, 最近: 2025-11-10
- 吞没形态 (CDLENGULFING) 出现 27 次, 最近: 2025-12-05
- 锤子线 (CDLHAMMER) 出现 9 次, 最近: 2025-12-10
- 早晨之星 (CDLMORNINGSTAR) 出现 3 次, 最近: 2025-08-29
- ...

看跌形态 (共 24 种出现过):

- 长实体 (CDLLONGLINE) 出现 63 次, 最近: 2025-12-31
- 射击之星 (CDLSHOOTINGSTAR) 出现 11 次, 最近: 2025-11-06
- 吞没形态 (CDLENGULFING) 出现 24 次, 最近: 2025-12-22
- ...

锤子线

锤子线 (Hammer) :

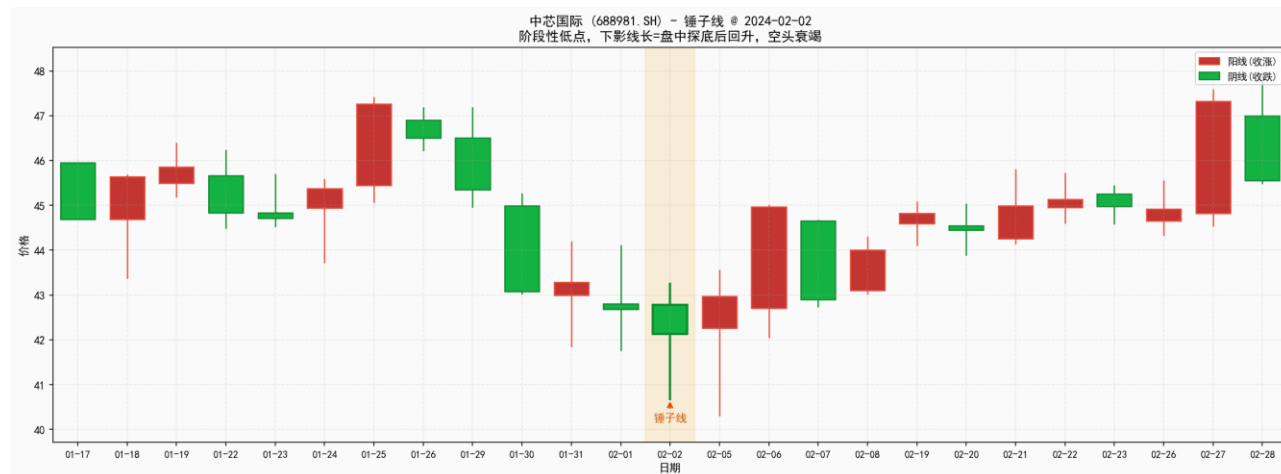
- 实体极短，下影线极长（通常规定下影线必须大于实体的 2 倍），几乎没有上影线。

- 资金博弈本质 (以底部的锤子线为例):

开盘后，恐慌盘和空头大举砸盘，价格雪崩式下跌（留下极深的下影线）。

但在盘中，主力资金或强力抄底盘介入，不仅吃掉了所有抛压，还将价格一路推升，最终收盘价甚至高于或逼近开盘价。

长下影线代表空头动能彻底衰竭，下方承接力极强。



射击之星

射击之星 (Shooting Star):

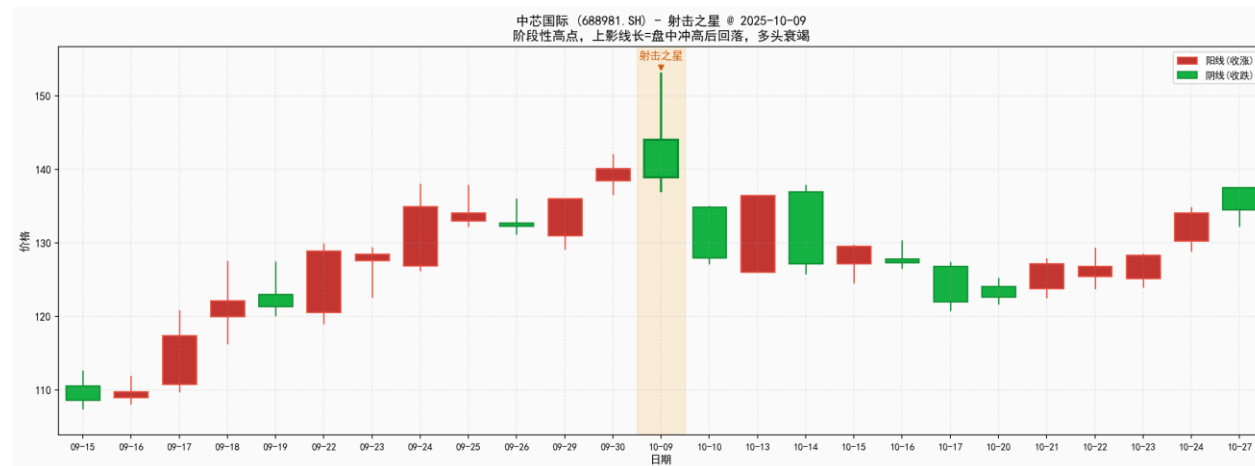
- 实体极短，上影线极长（通常上影线大于实体的2倍），几乎没有下影线。

- 资金博弈本质：

开盘后，多头惯性冲高，跟风盘涌入，价格一度大幅拉升（留下极高的上影线）。

但在盘中，主力资金开始高位派发，空头反击，不仅消化了所有买盘，还将价格一路打压。

长上影线代表多头动能彻底衰竭，上方抛压沉重，趋势即将反转。



(注：与锤子线相反，射击之星的实体位于整个价格区间的下端，且通常出现在上涨趋势的顶部)

吞没

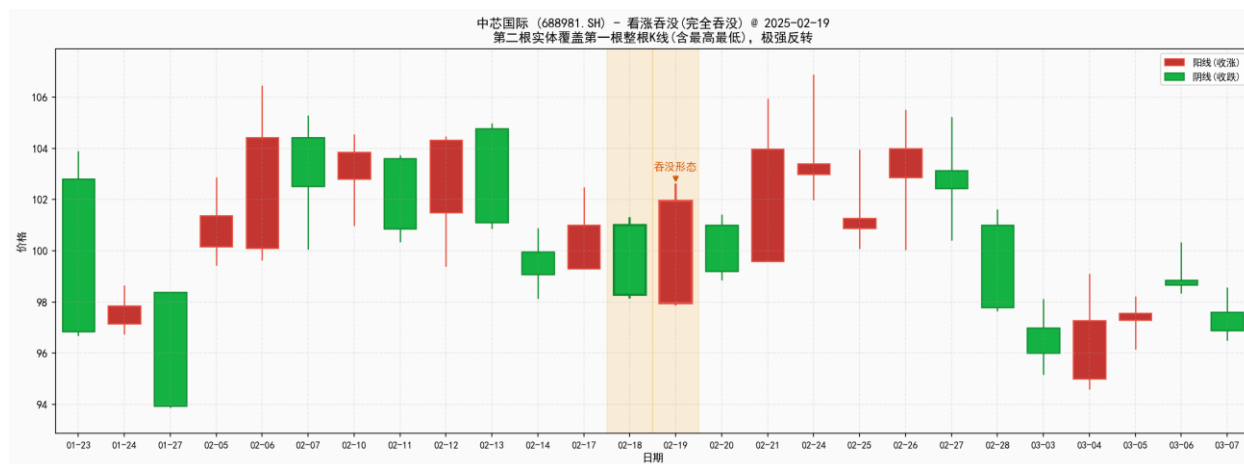
吞没形态 (Engulfing)

- 由两根 K 线组成。第二根 K 线的实体，完全包裹住第一根 K 线的实体。

- 资金博弈本质 (以看涨吞没/阳包阴为例):

昨日市场还在恐慌下跌收阴线。今日市场略微低开 (延续恐慌)，但随后多头彻底爆发，巨量资金推升价格，最终收盘价不仅越过了昨日的开盘价，还将昨日所有的套牢盘全部解放。

出现在下跌趋势末端，是高胜率的反转确立信号。



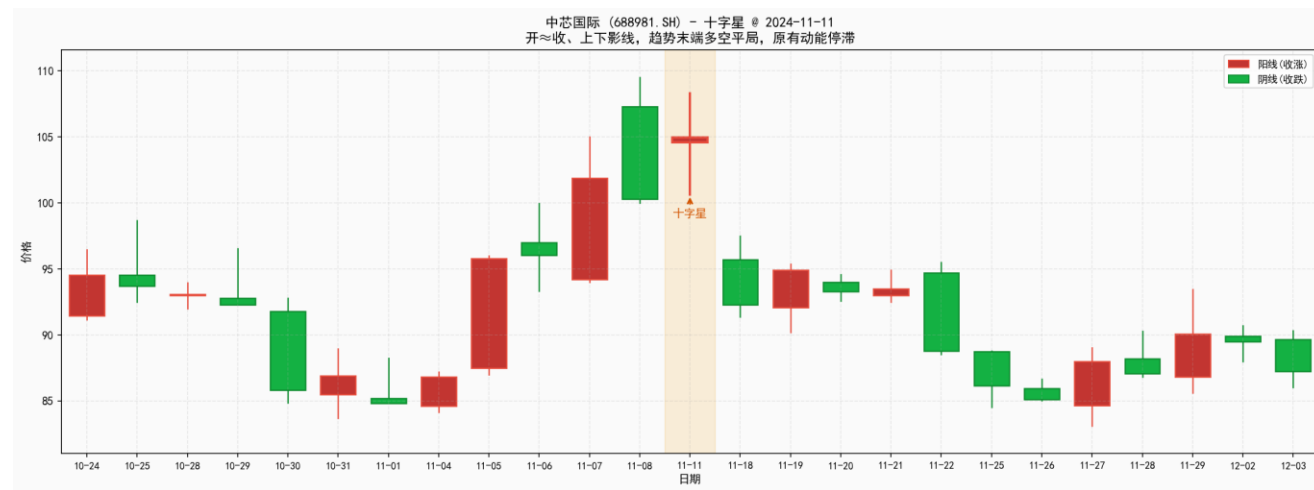
普通吞没：第二天实体仅覆盖第一天实体 => 中强

完全吞没：第二天实体覆盖第一天整个K线区间 => 极强

十字星

十字星 (Doji) :

- 开盘价等于（或极度接近）收盘价，带有上下影线。代表多空双方经过一天的激烈交战，以平局收场，原有趋势动能停滞。
- 十字星本身是中性的，只有在趋势末端才构成反转信号；如果在趋势中出现 => 中继整理



该日处于上涨末端，多空拉锯后收出十字星，随后出现明显下跌 => 趋势末端反转的十字星

早晨之星

早晨之星 (Morning Star) :

- 大阴线 (恐慌释放) => 十字星 (多空平衡、跌不动了) => 大阳线 (多头确认反攻) 。

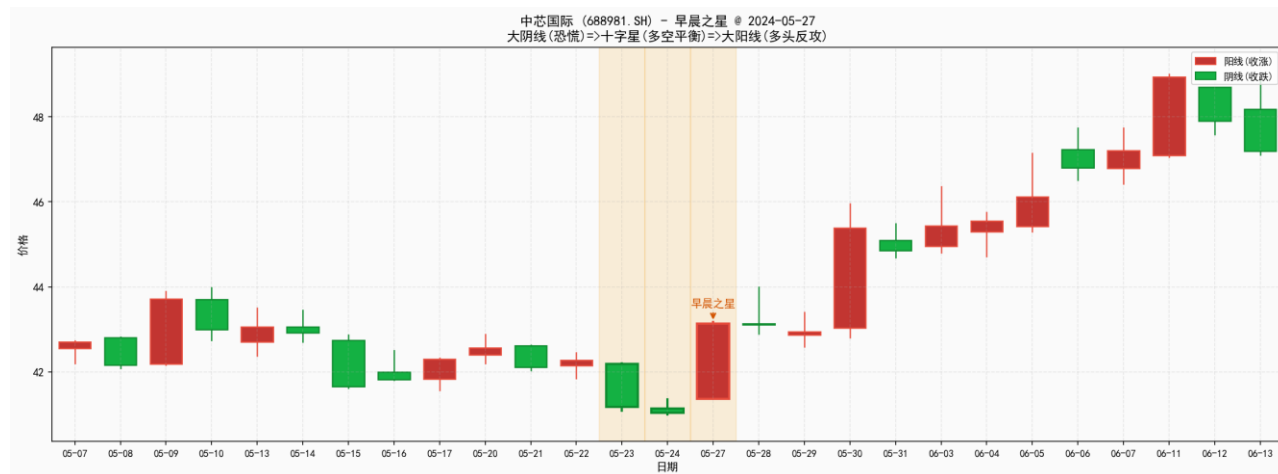
教科书级筹码换手周期

- 资金博弈本质:

第一日：散户恐慌割肉，主力暗中观察

第二日：主力开始试盘吸筹，多空达到短暂平衡
(跌无可跌)

第三日：主力完成布局，发动总攻，价格拉升同时
倒逼空头回补 (空头回补成为上涨燃料)



形态学在量化系统中的使用

Thinking: 形态学能否单独使用, 比如可以写一个策略: 只要出现锤子线就全仓买入?

回测结果通常是不好的, 因为在漫长的下跌阴跌中, 会出现无数个毫无意义的假锤子线。

=>形态学不能作为独立策略, 需要作为多维共振的过滤器。

场景 1: 精准捕捉均值回归 (结合布林带)

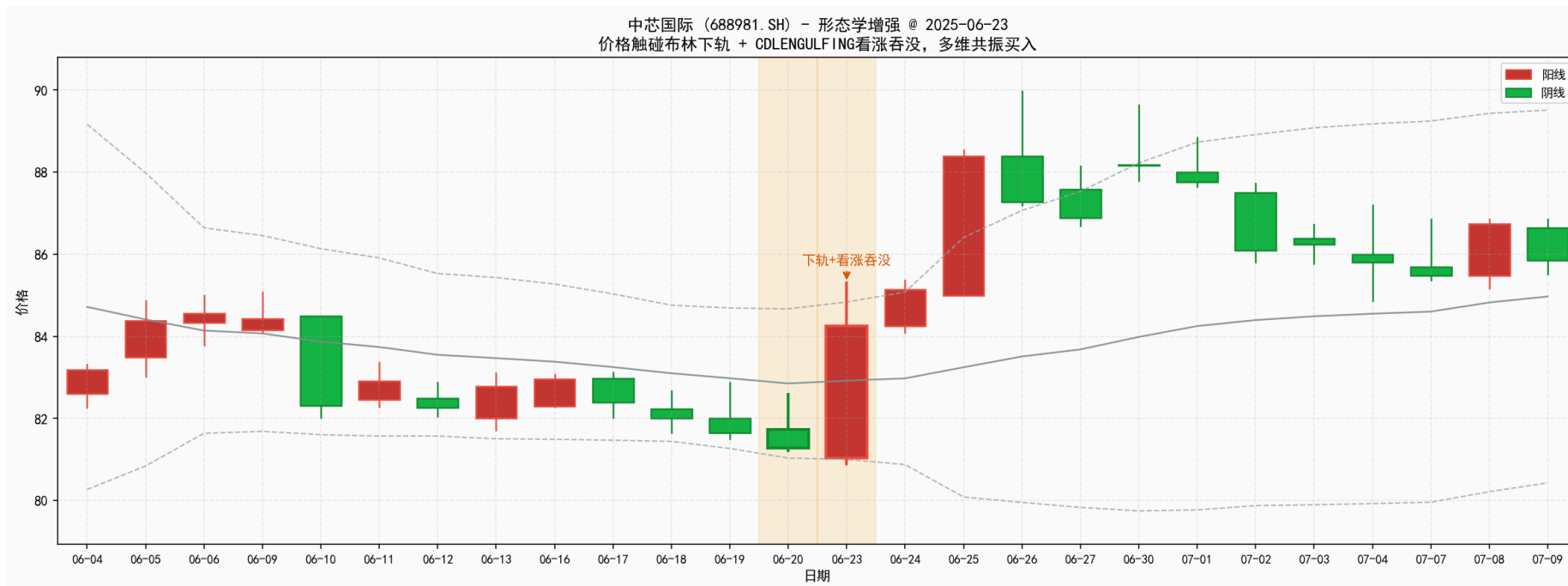
场景 2: 趋势回调买点 (结合均线)

场景 3: 横向选股雷达 (截面扫描)

场景 1：精准捕捉均值回归（结合布林带）

场景 1：精准捕捉均值回归（结合布林带）

- 常规逻辑：价格触碰布林带下轨买入（容易接飞刀被套）。
- 形态学增强：价格触碰布林带下轨，且当天 TA-Lib 识别出 锤子线 或 看涨吞没 返回 +100 时
=> 执行买入（相当于要求不仅价格到了低位，还需要看到资金在此处打出防守反击的痕迹）



场景 2：趋势回调买点（结合均线）

场景 2：趋势回调买点（结合均线）

- 常规逻辑：只做处于 60 日均线以上的股票。
- 形态学增强：股票处于 60 日均线多头排列中，短期回调至 20 日均线附近时，TA-Lib 扫描出 十字星 或 早晨之星 => 代表回调的下跌动能枯竭，趋势即将重启。

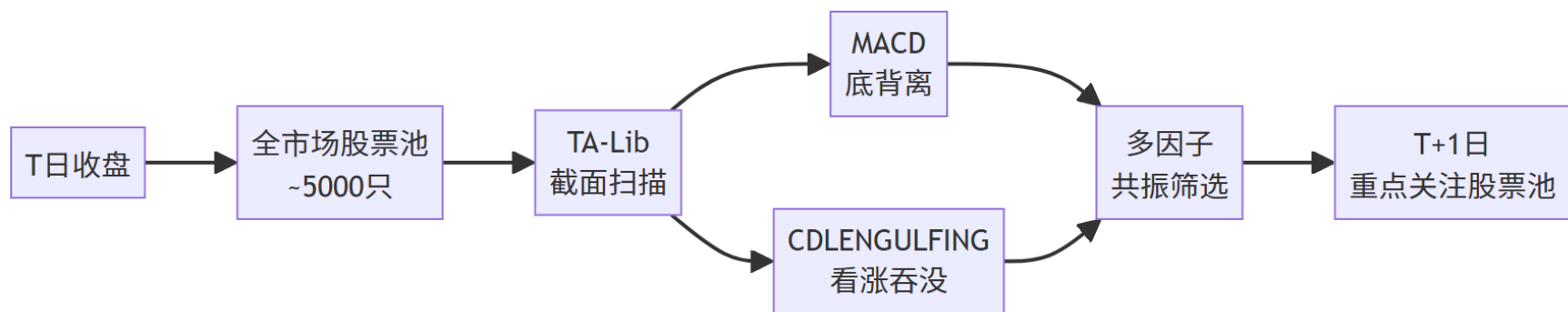


场景 3：横向选股雷达（截面扫描）

场景 3：横向选股雷达（截面扫描）

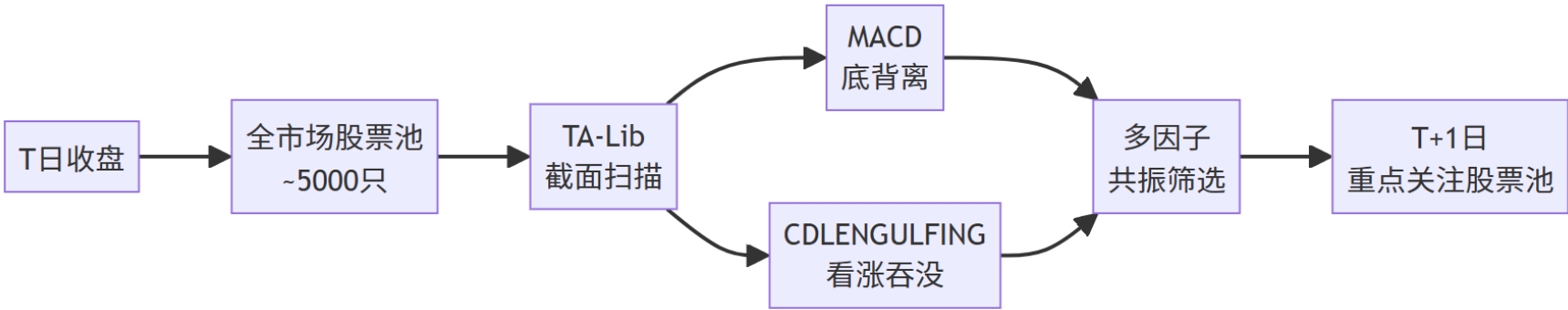
每天收盘后对全市场 5000 只股票运行一次 TA-Lib 形态扫描，找出当天同时满足 MACD 底背离且 K 线收出底部吞没形态的个股，作为次日重点关注的股票池。

=> 机构量化团队每天在做的特征工程提取。



场景 3：横向选股雷达（截面扫描）

股票代码	股票名称	MACD底背离	吞没形态	当日收盘	备注
688981. SH	中芯国际	是	是	112. 25	共振
600519. SH	贵州茅台	是	否	1415	仅背离
000858. SZ	五粮液	否	是	128. 5	仅形态
002594. SZ	比亚迪	是	是	245. 3	共振
...	...	...	...	...	...



传统指标增强

Thinking: 如何针对传统指标的缺陷进行增强?

量化投研，不是去发明一个前所未有的新指标，而是在经典指标的盲区中，加上符合市场博弈逻辑的过滤器（Filter）和风控阀门（Risk Control）。

1. 痛点一：RSI 抄底变徒手接飞刀
2. 痛点二：MACD 的缩量假金叉骗局
3. 痛点三：趋势跟踪策略的利润大出血
4. 痛点四：布林带的反弹夭折（死猫跳）陷阱

痛点一：RSI 抄底变徒手接飞刀

- **致命伤：**标准 RSI 策略教导我们，当 $RSI < 30$ 时进入超卖区，代表价格被低估，应该立刻买入。但在极端的单边下跌市（主跌浪）中，RSI 会在 20 甚至 10 以下持续钝化，价格可能在你买入后继续腰斩 => 此时的超卖不是机会，而是市场极度恐慌的体现。
- **实战优化：**穿越确认机制

不要在 RSI 跌破 30 的瞬间买入，而是把跌破 30 视为预警信号（将其纳入观察池）。

=> 当 RSI 跌落底部后，重新向上反弹并穿过 30 这条线时，才执行买入。

CASE: RSI策略-优化(穿越确认)

```
class RSISandard(bt.Strategy):
    """标准版: RSI < 30 立即买入"""
    params = (('period', 14), ('oversold', 30), ('overbought', 70))

    def __init__(self):
        self.rsi = bt.indicators.RSI(self.data.close, period=self.p.period)

    def next(self):
        if not self.position:
            if self.rsi[0] < self.p.oversold: # 跌破30立即买 -> 可能接飞刀
                self.buy()
        elif self.rsi[0] > self.p.overbought:
            self.close()
```

```
class RSIOptimized(bt.Strategy):
    """优化版: talib.RSI + 穿越确认"""
    params = (('period', 14), ('oversold', 30), ('overbought', 70))

    def _calc_rsi(self):
        size = len(self.data)
        close = np.array(self.data.close.get(size=size), dtype=np.float64)
        return talib.RSI(close, timeperiod=self.p.period)

    def next(self):
        if len(self.data) < self.p.period + 2:
            return
        rsi = self._calc_rsi()
        if not self.position:
            # 穿越确认: RSI从超卖区回升穿过30线, 确认止跌反弹才买入
            if rsi[-2] < self.p.oversold and rsi[-1] >= self.p.oversold:
                self.buy()
        else:
            if rsi[-1] > self.p.overbought:
                self.close()
```

4-RSI策略-优化(穿越确认).py

CASE：RSI策略-优化(穿越确认)

博弈本质：rsi[-2] < 30 and rsi[-1] >= 30

代表空方力量已经彻底释放完毕，多方资金开始实打实地进场扫货，将动能重新拉回安全线上方

=> 减少接飞刀的概率，降低策略的最大回撤并提升夏普比率。

标的	标准版收益	优化版收益	标准版回撤	优化版回撤
贵州茅台	20.55%	23.31%	4.39%	2.45%
平安银行	12.92%	12.03%	4.18%	4.18%
纳指ETF	1.85%	3.05%	16.88%	15.85%

痛点二：MACD 的缩量假金叉骗局

- **致命伤：**MACD 标准策略是快线（DIF）上穿慢线（DEA）即全仓买入。但 MACD 的计算公式完全只依赖价格，忽略了市场的成交量。在低迷的震荡市中，主力只需极少的资金就能在尾盘把价格拉高，人造出一个完美的 MACD 金叉（骗线）
- **实战优化（成交量均线验证）：**

所有的价格突破，如果没有成交量的配合，都缺乏可靠性。

=> 引入 `talib.SMA(volume, 20)` 计算过去 20 天的平均成交量，作为金叉的有效性验证。

当 MACD 发生金叉时，强制要求当天的成交量必须大于 20 日均量的 90%（甚至 100%），

即 $\text{volume}[-1] > \text{vol_ma}[-1] * 0.9$ 。

CASE: MACD策略-优化(成交量确认)

```
class MACDOptimized(bt.Strategy):
    """优化版: talib.MACD + 成交量确认入场"""
    params = (('fast', 12), ('slow', 26), ('signal', 9),
              ('vol_period', 20), ('vol_mult', 0.9))

    def _calc(self):
        """用talib计算MACD和成交量均线"""
        size = len(self.data)
        close = np.array(self.data.close.get(size=size), dtype=np.float64)
        volume = np.array(self.data.volume.get(size=size), dtype=np.float64)
        macd, signal, hist = talib.MACD(close,
            fastperiod=self.p.fast, slowperiod=self.p.slow, signalperiod=self.p.signal)
        vol_ma = talib.SMA(volume, timeperiod=self.p.vol_period)
        return macd, signal, volume, vol_ma

    def _is_golden_cross(self, macd, signal):
        if len(macd) < 2 or np.isnan(macd[-1]) or np.isnan(macd[-2]):
            return False
        return macd[-2] <= signal[-2] and macd[-1] > signal[-1]
```

```
    def _is_dead_cross(self, macd, signal):
        # ... (同理判断死叉)

    def next(self):
        if len(self.data) < self.p.slow + self.p.signal:
            return
        macd, signal, volume, vol_ma = self._calc()

        if not self.position:
            if self._is_golden_cross(macd, signal):
                # 成交量确认: 当日成交量必须大于 20 日均量的 90%
                vol_ok = not np.isnan(vol_ma[-1]) and volume[-1] > vol_ma[-1] *
self.p.vol_mult
                if vol_ok:
                    self.buy()
            else:
                if self._is_dead_cross(macd, signal):
                    self.close()
```

5-MACD策略-优化(成交量确认).py

CASE：MACD策略-优化(成交量确认)

过滤掉缺乏资金真实推动的缩量金叉假信号。量价齐升的金叉 => 才是趋势启动的真正标志。

标的	标准版收益	优化版收益	标准版回撤	优化版回撤
贵州茅台	-4.56%	-4.56%	9.14%	9.14%
中芯国际	17.91%	16.64%	14.23%	14.23%
平安银行	15.21%	-0.26%	6.16%	6.07%
纳指ETF	-3.43%	6.05%	18.72%	4.12%
平均	6.28%	4.47%	12.06%	8.39%

成交量过滤在纳指ETF上效果显著: 收益 -3.43% -> +6.05%, 回撤 18.72% -> 4.12%。

平均回撤从 12.06% 降至 8.39%。

痛点三：趋势跟踪策略的利润大出血

- **致命伤**：无论是 MACD 策略（等死叉卖出）还是海龟交易法则（等跌破 10 日低点卖出），都属于趋势跟踪策略。这类策略最大的痛苦在于：卖出信号极其滞后。当 MACD 真正形成死叉时，价格往往已经从最高点暴跌了 10% 甚至 20% => 导致账面利润大幅回吐

- **实战优化（动态利润锁定）**：

抛弃机械的死叉离场，改用基于盈亏比例的动态跟踪止盈。

设定两个阈值：例如 `profit_trigger = 5.0%` 和 `trail_pct = 3.0%`。

当持仓盈利未达到 5% 时，依然依赖底层的指标逻辑（如 MACD 死叉止损）。

一旦持仓盈利越过 5% 的激活线，系统开始只记录最高价。

只要价格从已创出的最高点回落超过 3%，无视 MACD 是否死叉，立刻强行平仓落袋为安。

CASE: MACD策略-优化(利润锁定)

```
class MACDProfitLock(bt.Strategy):
    """优化版: talib.MACD + 利润锁定出场"""
    params = (('fast', 12), ('slow', 26), ('signal', 9),
              ('profit_trigger', 5.0), ('trail_pct', 3.0))

    def __init__(self):
        self._entry_price = 0.0
        self._peak_price = 0.0

    def _calc_macd(self):
        size = len(self.data)
        close = np.array(self.data.close.get(size=size), dtype=np.float64)
        macd, signal, hist = talib.MACD(close,
                                         fastperiod=self.p.fast, slowperiod=self.p.slow, signalperiod=self.p.signal)
        return macd, signal

    # ... (_is_golden_cross / _is_dead_cross 同上)
```

```
def next(self):
    if len(self.data) < self.p.slow + self.p.signal:
        return
    macd, signal = self._calc_macd()
    if not self.position:
        if self._is_golden_cross(macd, signal):
            self.buy()
            self._entry_price = self.data.close[0]
            self._peak_price = self.data.close[0]
        else:
            # 持仓期间持续跟踪最高价
            self._peak_price = max(self._peak_price, self.data.close[0])
            gain = (self._peak_price - self._entry_price) / self._entry_price * 100
            drop = (self._peak_price - self.data.close[0]) / self._peak_price * 100
            # 盈利越过5%后, 从高点回落超过3%即平仓落袋
            if gain >= self.p.profit_trigger and drop >= self.p.trail_pct:
                self.close()
            elif self._is_dead_cross(macd, signal):
                self.close() # 未达激活线时, 仍依赖死叉离场
```

6-MACD策略-优化(利润锁定).py

CASE：MACD策略-优化(利润锁定)

MACD利润锁定回测结果（2025年）：

标的	标准版收益	优化版收益	标准版回撤	优化版回撤
贵州茅台	-4.56%	-2.96%	9.14%	9.14%
中芯国际	17.91%	26.14%	14.23%	12.20%
平安银行	15.21%	14.98%	6.16%	5.30%
纳指ETF	-3.43%	-4.87%	18.72%	18.72%
平均	6.28%	8.32%	12.06%	11.34%

中芯国际效果最为显著: 收益 +17.91% -> +26.14%, 回撤 14.23% -> 12.20%, 夏普 1.31 -> 2.23。

痛点四：布林带的反弹夭折（死猫跳）陷阱

- **致命伤**：标准布林带策略认为，价格触碰下轨买入，触碰上轨卖出。但在弱势市场中，价格触碰下轨后，往往只是产生微弱的反弹，连上轨的边都摸不到，就再次掉头向下，开启新一轮暴跌（即沿着下轨滑滑梯）。如果死等上轨卖出 => 会导致深度套牢。
- **实战优化（中轨防守反击止损法）**：

布林带的中轨（通常是 20 日均线）是多空力量的强弱分水岭。

在下轨买入后，设置一个状态机标签 `_bounced`。

当价格反弹并越过中轨时，标记 `_bounced = True`（确认反弹有效）。

在此之后，如果价格未能继续向上轨进发，反而掉头跌破了中轨，立刻清仓止损。

死猫跳是股市中的技术术语，即下跌趋势中的短暂技术性反弹——就像死猫从高处摔下也会弹起一下，但终究难逃一死。

CASE：布林带策略-优化(中轨止损)

```
class BollingerOptimized(bt.Strategy):
    """优化版: talib.BBANDS + 中轨止损"""
    params = (('period', 20), ('dev', 2.0))

    def __init__(self):
        self._bounced = False    # 状态机: 是否已反弹到中轨以上

    def _calc_bbands(self):
        """用talib计算布林带"""
        size = len(self.data)
        close = np.array(self.data.close.get(size=size), dtype=np.float64)
        upper, middle, lower = talib.BBANDS(close,
            timeperiod=self.p.period, nbdevup=self.p.dev, nbdevdn=self.p.dev)
        return upper, middle, lower

    def next(self):
        if len(self.data) < self.p.period + 1:
            return

        upper, middle, lower = self._calc_bbands()
```

```
        if not self.position:
            if self.data.close[0] <= lower[-1]: # 触及下轨买入
                self.buy()
                self._bounced = False

            else:
                if self.data.close[0] > middle[-1]:
                    self._bounced = True    # 价格越过中轨 -> 确认反弹有效

                    if self._bounced and self.data.close[0] < middle[-1]:
                        self.close()    # 反弹有效后又跌破中轨 -> 反弹失败止损
                    elif self.data.close[0] >= upper[-1]:
                        self.close()    # 触及上轨 -> 正常止盈
```

7-布林带策略-优化(中轨止损).py

痛点四：布林带的反弹夭折（死猫跳）陷阱

博弈本质: 设置状态机标签 `_bounced`，将出场分为两种情况:

- 反弹成功: 价格越过中轨后继续上行到上轨，正常止盈
- 反弹失败: 价格越过中轨后未能持续上攻、又跌破中轨，及时止损，避免二次下跌的更大损失

标的	标准版收益	优化版收益	标准版回撤	优化版回撤
贵州茅台	7.11%	5.29%	2.49%	2.35%
中芯国际	-1.55%	-1.01%	11.91%	8.07%
平安银行	3.09%	4.23%	8.64%	6.21%
纳指ETF	1.19%	5.57%	17.94%	8.98%
标的	标准版收益	优化版收益	标准版回撤	优化版回撤

中轨止损在高波动标的上效果显著: 纳指ETF收益 +1.19% -> +5.57%, 回撤 17.94% -> 8.98%。

CASE：自适应策略

金融市场并非均匀分布的。统计学表明，市场大约有 30% 的时间处于明显的趋势中（单边上涨或下跌）
70% 的时间处于无方向的震荡洗盘中。

Thinking：如果用应对 30% 行情的 MACD 去跑那 70% 的震荡行情 => 遭遇资金曲线的严重回撤。

有什么好的办法？

我们在执行买卖指令前，可以先问一个问题：当前市场是趋势市，还是震荡市？

如果能更好的了解当前的环境 => 可以选择适合的交易策略。

ADX：平均趋向指数

为了判断市场环境，引入 TA-Lib 的重量级算子：talib.ADX

- 绝大多数新手认为指标都是用来判断涨跌的，但 ADX 根本不判断涨跌方向，它只测量趋势的拥挤度（趋势强度）。当行情暴涨时，ADX 会飙升；
- 当行情暴跌时，ADX 同样会飙升（说明下跌趋势极强）。
- 只有当多空双方陷入胶着、价格横盘画门时，ADX 才会萎靡不振地趴在低位。

CASE：自适应策略

TO DO：将前几个模块学到的打好补丁的策略组合起来，形成一个智能调度引擎：

状态一：趋势市 —— $ADX > 25$

- 环境判断：市场进入了明显的单边行情，动能充沛。
- 策略激活：引擎自动屏蔽 RSI 等震荡指标，激活 MACD 策略。
- 执行逻辑：耐心等待 MACD 发生金叉（动能向上发散）时顺势买入。只要趋势未被打破（未出现 MACD 死叉），就死死咬住利润不松口。

CASE：自适应策略

状态二：震荡市—— $ADX < 20$

- 环境判断：市场失去了方向，处于上有阻力、下有支撑的箱体震荡中。

如果此时用 MACD => 会被频繁的假金叉/假死叉绞杀。

- 策略激活：引擎自动锁定 MACD，唤醒 RSI 均值回归策略。
- 执行逻辑：启动高抛低吸模式。

当 $RSI < 30$ 且出现超卖反弹迹象时买入；

当 $RSI > 70$ 触及箱体顶部时，果断止盈，绝不贪恋（因为震荡市中很难有主升浪）。

CASE：自适应策略

状态三：混沌过渡区 —— $20 \leq ADX \leq 25$

- 环境判断：旧的趋势正在瓦解，新的方向尚未明朗，这是胜率最低的绞肉机行情。
- 执行逻辑：系统进入观望状态，不发出任何买入指令。

在量化交易中，空仓本身就是重要的防守策略。

CASE：自适应策略

```
class AdaptiveStrategy(bt.Strategy):
    params = (
        ('adx_period', 14),
        ('adx_trend', 25), ('adx_range', 20),
        ('atr_period', 14), ('atr_mult', 2.0),
        ('macd_fast', 12), ('macd_slow', 26), ('macd_signal', 9),
        ('rsi_period', 14),
    )
    def __init__(self):
        self._stop_price = 0.0
        self._mode = None
    def _calc_indicators(self):
        """用talib计算ADX、MACD、RSI、ATR"""
        size = len(self.data)
        high = np.array(self.data.high.get(size=size), dtype=np.float64)
        low = np.array(self.data.low.get(size=size), dtype=np.float64)
        close = np.array(self.data.close.get(size=size), dtype=np.float64)
```

```
        adx = talib.ADX(high, low, close, timeperiod=self.p.adx_period)
        macd, signal, _ = talib.MACD(close,
            fastperiod=self.p.macd_fast, slowperiod=self.p.macd_slow,
            signalperiod=self.p.macd_signal)
        rsi = talib.RSI(close, timeperiod=self.p.rsi_period)
        atr = talib.ATR(high, low, close, timeperiod=self.p.atr_period)
        return adx, macd, signal, rsi, atr
    # ... (_is_golden_cross / _is_dead_cross 同前)

    def next(self):
        if len(self.data) < self.p.macd_slow + self.p.macd_signal:
            return
        adx, macd, signal, rsi, atr = self._calc_indicators()
        # --- 判断市场状态 ---
        adx_val = adx[-1]
        if np.isnan(adx_val):
            return
```

CASE：自适应策略

```
if adx_val > self.p.adx_trend:
    regime = 'trend'    # 趋势市 -> MACD
elif adx_val < self.p.adx_range:
    regime = 'range'    # 震荡市 -> RSI
else:
    regime = 'neutral'  # 过渡区 -> 观望
atr_val = atr[-1] if not np.isnan(atr[-1]) else 0.0
if not self.position:
    # 趋势市: MACD 金叉买入
    if regime == 'trend' and self._is_golden_cross(macd, signal):
        self.buy()
        self._stop_price = self.data.close[0] - self.p.atr_mult * atr_val
        self._mode = 'trend'
    # 震荡市: RSI 超卖买入
    elif regime == 'range' and not np.isnan(rsi[-1]) and rsi[-1] < 30:
        self.buy()
        self._stop_price = self.data.close[0] - self.p.atr_mult * atr_val
        self._mode = 'range'
```

```
else:
    # ATR 跟踪止损 (棘轮: 只升不降)
    new_stop = self.data.close[0] - self.p.atr_mult * atr_val
    self._stop_price = max(self._stop_price, new_stop)
    stop_hit = self.data.close[0] < self._stop_price

    if self._mode == 'trend':
        if self._is_dead_cross(macd, signal) or stop_hit:
            self.close()
            self._mode = None
    elif self._mode == 'range':
        if (not np.isnan(rsi[-1]) and rsi[-1] > 70) or stop_hit:
            self.close()
            self._mode = None
```

ATR 动态吊灯止损

在自适应策略中，由于我们会在不同策略间切换，不能再依赖单一指标自带的死叉来离场。为此，我们引入 talib.ATR（真实波幅），在底层建立统一的动态止损线：

- **动态计算**：每当建仓后，系统会实时计算一个动态止损价： $StopPrice = P_{close} - k \times ATR_n$
- **棘轮效应（只进不退）**：随着价格的上涨，这个止损价会被不断向上拖动（`self._stop_price = max(self._stop_price, new_stop)`）。但只要价格回调，止损价绝不降低。一旦收盘价跌破这个动态底线，无论当前处于什么状态、MACD 有没有死叉，引擎都会触发无条件强制平仓。

优势：在低迷行情中，ATR 较小，止损线收紧，防止钝刀子割肉；

在主升浪中，ATR 放大，止损线自动放宽，给予趋势充分的震荡洗盘空间，防止被主力轻易洗下车。

CASE：自适应策略

自适应策略 vs 纯MACD策略

ADX市场状态判断:

ADX > 25: 趋势市(用MACD跟踪趋势)

ADX < 20: 震荡市(用RSI抄底逃顶)

20-25: 过渡区(观望)

[纯MACD] 不分市场环境:

纯MACD | 600519.SH | 2024-01-02 ~ 2025-12-31 | 481个交易日

总收益: -9.05% | 年化: -4.85% | 最大回撤: 22.52% | 夏普: -1.67 | 卡玛: -0.22

交易: 15次 | 胜率: 26.7% | 盈亏比: 1.75 | 利润因子: 0.70 | 最大连亏: 3次

图表已保存: outputs\纯MACD.png

[自适应] ADX状态切换:

自适应策略 | 600519.SH | 2024-01-02 ~ 2025-12-31 | 481个交易日

总收益: +14.17% | 年化: +7.19% | 最大回撤: 10.96% | 夏普: 0.88 | 卡玛: 0.66

交易: 5次 | 胜率: 40.0% | 盈亏比: 2.91 | 利润因子: 1.94 | 最大连亏: 2次

自适应策略显著优于纯MACD:

收益从 -9.05% 提升到 +14.17%，回撤从 22.52% 降至 10.96%，交易次数从 15 次减少到 5 次，胜率从 26.7% 提升到 40.0%

Summary

- **TA-Lib定位：** C语言底层的高性能技术分析库，是连接传统技术分析与现代AI量化的桥梁（Array In → Array Out, 无缝对接ML特征工程）
- **性能优势：** 计算速度比Pandas快29倍（全市场5000只股票截面分析必备）

- **经典指标盲区：**

均线/MACD等滞后指标存在信息折损；

RSI在单边市会钝化（<20仍可能持续下跌）；

布林带可能遭遇死猫跳（触及下轨后反弹夭折）

Summary

- 形态学价值:

61种CDL形态（锤子线/吞没等）捕捉当下资金博弈，属于左侧/同步信号，弥补滞后指标缺陷

核心函数速查

talib.SMA/EMA/WMA # 均线类（趋势轨道）

talib.MACD # 动量类（加速度计算：DIF-DEA）

talib.RSI # 情绪类（0-100极端区）

talib.ATR # 波动率类（动态止损基准）

talib.BBANDS # 通道类（中轨为多空分水岭）

talib.ADX # 环境判断类（趋势强度，非方向）

talib.CDLHAMMER/CDLENGULFING # 61种形态学（输出+100/-100/0）

Summary

形态名	TA-Lib 函数名	输出信号	特征
锤子线	CDLHAMMER	100	实体极短，下影线极长（>2倍实体），出现在下跌趋势底部
射击之星	CDLSHOOTINGSTAR	-100	实体极短，上影线极长（>2倍实体），出现在上涨趋势顶部
吞没形态	CDLENGULFING	-1	第二根 K 线实体完全包裹第一根（阳包阴看涨/阴包阳看跌）
十字星	CDLDOJI	0（中性）	开收盘几乎相等，带有上下影线，趋势末端才具反转意义
早晨之星	CDLMORNINGSTAR	100	三根 K 线组合：大阴→十字星/小实体→大阳（教科书级底部反转）

Summary

Thinking: 除了这5种常用的形态, 实战中常用的还有哪些 (基于 TA-Lib 标准库) 形态? :

- 反转形态类:

CDLEVENINGSTAR - 黄昏之星 (早晨之星的顶部版本, 看跌)

CDLPIERCING - 刺透形态/曙光初现 (下跌中阳线刺入阴线实体 50% 以上)

CDLDARKCLOUDCOVER - 乌云盖顶 (上涨中阴线刺入阳线实体 50% 以下)

CDLINVERTEDHAMMER - 倒锤子线 (出现在底部, 上影线长, 实体在最低价附近)

CDLHARAMI - 孕线 (第二根 K 线实体完全在第一根之内, 趋势停滞信号)

CDLHARAMICROSS - 十字孕线 (孕线+十字星, 更强的变盘信号)

- 持续/确认形态类:

CDL3WHITESOLDIERS - 白三兵/三红兵 (三根逐步升高的阳线, 强势看涨持续)

CDL3BLACKCROWS - 三只乌鸦 (三根逐步降低的阴线, 强势看跌持续)

CDLMARUBOZU - 光头光脚/秃头蜡烛 (无上下影线, 极强单边力量)

CDLTASUKIGAP - 跳空并列阴阳线 (跳空缺口后的整理形态)

- 特殊十字星:

CDLDRAGONFLYDOJI - 蜻蜓十字星 (T 字形, 下影线长, 开盘收盘在最高价, 强烈看涨)

CDLGRAVESTONEDOJI - 墓碑十字星 (倒 T 字形, 上影线长, 开盘收盘在最低价, 强烈看跌)

Summary

TA-Lib的形态函数：

```
import talib

# 1. 所有CDL函数列表（共61种）
# =====
cdl_funcs = [f for f in talib.get_functions() if f.startswith('CDL')]
print(f"\nTA-Lib K线形态函数: {len(cdl_funcs)} 种")
print("完整列表:")
for i, f in enumerate(cdl_funcs):
    print(f" {i+1:2d}. {f}")
```

TA-Lib K线形态函数: 61 种
完整列表:

1. CDL2CROWS
2. CDL3BLACKCROWS
3. CDL3INSIDE
4. CDL3LINESTRIKE
5. CDL3OUTSIDE
6. CDL3STARSINSOUTH
7. CDL3WHITESOLDIERS
8. CDLABANDONEDBABY
9. CDLADVANCEBLOCK
10. CDLBELTHOLD
11. CDLBREAKAWAY
12. CDLCLOSINGMARUBOZU
- ...

打卡：制定你的股票2025年交易策略



你的自选股票，在2025年采用哪种交易策略是适合的

- 可以使用自选股，也可以使用这里的 贵州茅台
- 经典指标的基础上，结合TA-Lib的独特因子，进行策略优化

CASE：RSI策略-优化(穿越确认)

CASE：MACD策略-优化(成交量确认)

CASE：MACD策略-优化(利润锁定)

CASE：布林带策略-优化(中轨止损)

CASE：自适应策略

- 你觉得哪种优化策略更适合你的自选股，原因是什么？



Thank You
Using data to solve problems